

Linked Lists-Applications

4.4 POLYNOMIAL ADDITION

POLYNOMIAL ADDITION

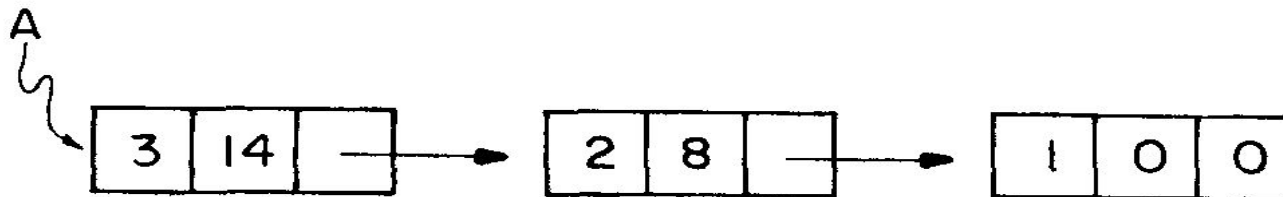
- Let us tackle a reasonable size problem **polynomial addition** by using linked lists.
- This problem, the manipulation of symbolic polynomials, has become a classical example of the use of list processing.

$$A(x) = a_m x^{e_m} + \dots + a_1 x^{e_1}$$

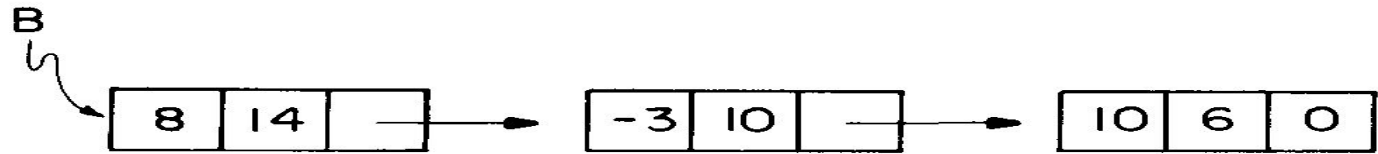
where the a_i are non-zero coefficients with exponents e_i such that $e_m > e_{m-1} > \dots > e_2 > e_1 \geq 0$. Each term will be represented by a node. A node will be of fixed size having 3 fields which represent the coefficient and exponent of a term plus a pointer to the next term



For instance, the polynomial $A = 3x^{14} + 2x^8 + 1$ would be stored as



while $B = 8x^{14} - 3x^{10} + 10x^6$ would look like



- In order to add two polynomials together we **examine** their **terms** starting at the nodes pointed to by **A** and **B**.
- **Two pointers p and q** are used to move along the terms of **A** and **B**.
- If the **exponents of two terms are equal**, then the coefficients are added and a new term created for the result.
- If the exponent of the current term in **A** is less than the exponent of the current term of **B**, then a duplicate of the term of **B** is created and attached to **C**.
- The pointer q is advanced to the next term.
- Similar action is taken on **A** if $\text{EXP}(p) > \text{EXP}(q)$.
- Figure 4.7 illustrates this addition process on the polynomials **A** and **B** above.

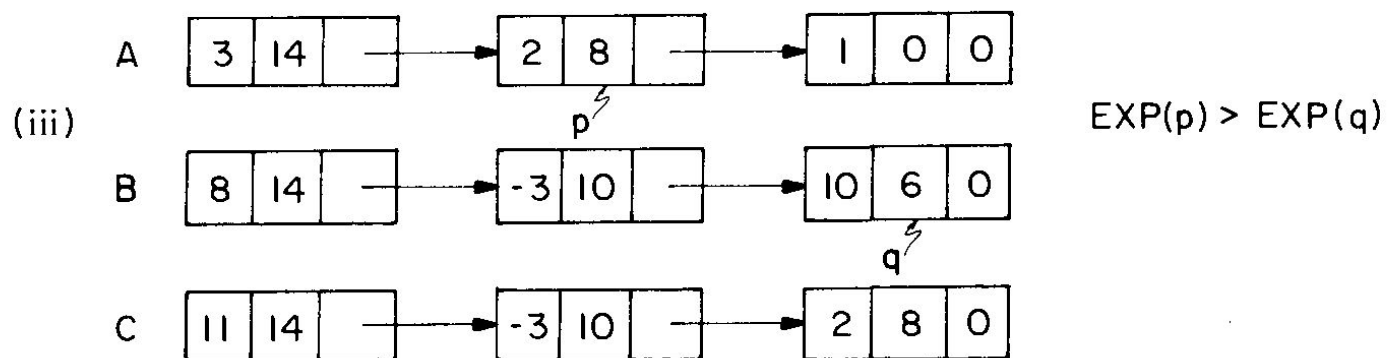
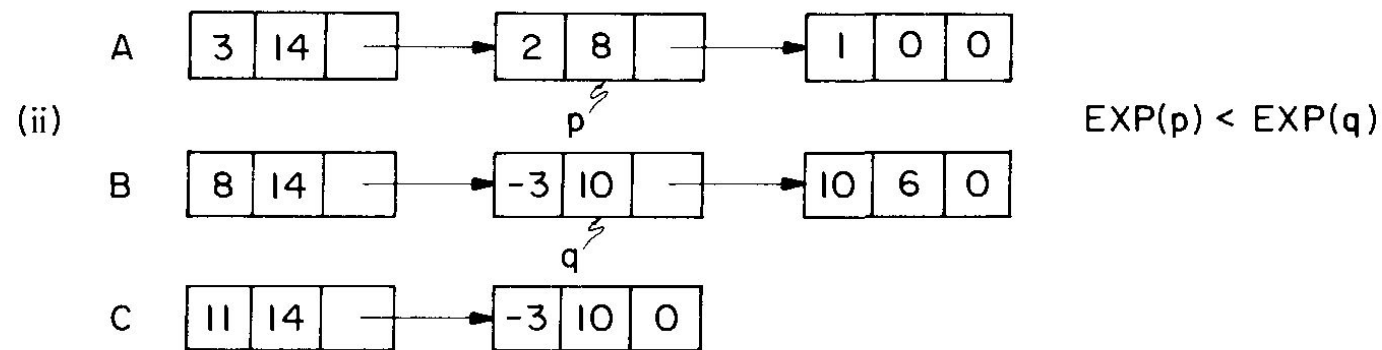
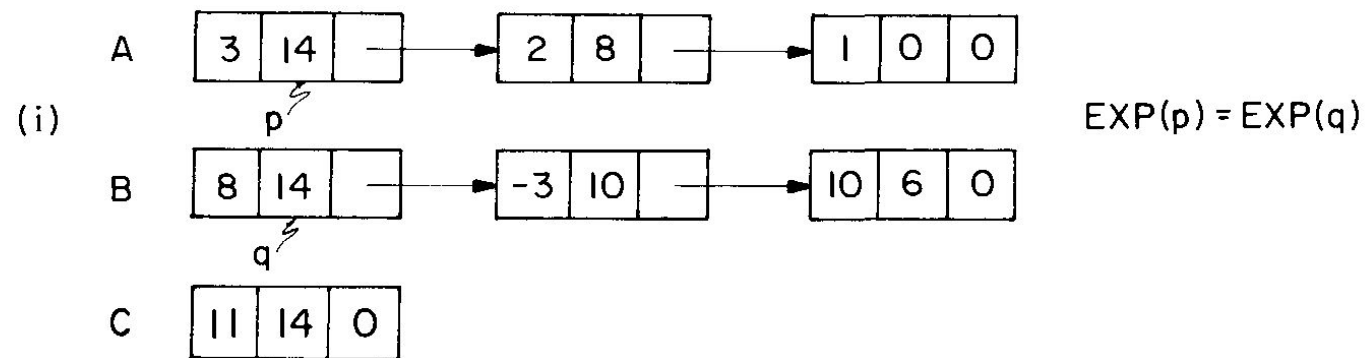


Figure 4.7 Generating the First Three Terms of $C = A + B$

- Each time a **new node** is generated its COEF and EXP fields are set and it is **appended** to the **end of the list C**.
- In order to **avoid** having to **search** for the last node in C each time a new node is added, we keep a pointer **d** which points to the **current last node** in C.
- The complete addition algorithm is specified by the procedure **PADD**.
- PADD makes use of a subroutine **ATTACH** which creates a new node and appends it to the end of C.
- To make things work out neatly, **C** is initially given a **single node with no values** which is deleted at the end of the algorithm.
- Though this is somewhat inelegant, it avoids more computation.

procedure *ATTACH*(*C*, *E*, *d*)

 //create a new term with $COEF = C$ and $EXP = E$ and attach it
 to the node pointed at by *d* //

 call *GETNODE*(*I*)

$EXP(I) \leftarrow E$

$COEF(I) \leftarrow C$

$LINK(d) \leftarrow I$ //attach this node to the end of this list //

$d \leftarrow I$ //move pointer *d* to the new last node //

end *ATTACH*

- The basic algorithm is straightforward, using a **merging process** which streams along the two polynomials either **copying terms** or **adding them** to the result.
- Thus, the main while loop of lines 3-15 has **3 cases** depending upon whether the next **pair of exponents** are **=**, **<**, or **>**.
- Notice that there are 5 places where a new term is created, justifying our use of the subroutine ATTACH.
- In order to carry out a computing time analysis it is first necessary to determine which operations contribute to the cost.
- For this algorithm there are several **cost measures**:
 - (i) coefficient additions;
 - (ii) exponent comparisons;
 - (iii) additions/deletions to available space;
 - (iv) creation of new nodes for C

procedure PADD(A, B, C)

 //polynomials A and B represented as singly linked lists are
 summed to form the new list named C//

1 $p \leftarrow A; q \leftarrow B$ //p,q pointers to next term of A, B//

2 **call** GETNODE(C); $d \leftarrow C$ //initial node for C, returned
 later//

3 **while** $p \neq 0$ **and** $q \neq 0$ **do** //while there are more terms in
 A and B//

4 **case**

5 : $EXP(p) = EXP(q)$: //equal exponents//

6 $x \leftarrow COEF(p) + COEF(q)$

7 **if** $x \neq 0$ **then call** ATTACH($x, EXP(p), d$)

8 $p \leftarrow LINK(p); q \leftarrow LINK(q)$ //advance to next
 terms//

9 : $EXP(p) < EXP(q)$:

10 **call** ATTACH($COEF(q), EXP(q), d$)

11 $q \leftarrow LINK(q)$ //advance to next term//

12 : **else** : **call** ATTACH($COEF(p), EXP(p), d$)

13 $p \leftarrow LINK(p)$ //advance to next term of A//

14 **end**

15 **end**

16 **while** $p \neq 0$ **do** //copy remaining terms of A//

17 **call** ATTACH($COEF(p), EXP(p), d$)

18 $p \leftarrow LINK(p)$

19 **end**

20 **while** $q \neq 0$ **do** //copy remaining terms of B//

21 **call** ATTACH($COEF(q), EXP(q), d$)

22 $q \leftarrow LINK(q)$

23 **end**

24 $LINK(d) \leftarrow 0; t \leftarrow C; C \leftarrow LINK(C)$ //delete extra initial
 node//

25 **call** RET(t)

26 **end PADD**

- Let us assume that each of these four operations, if done once, takes a single unit of time.
- The total time taken by algorithm PADD is then determined by the **number of times** these **operations** are performed.
- This number clearly depends on how many **terms** are present in the polynomials A and B .
- Assume that A and B have m and n terms respectively.

$$A(x) = a_m x^{e_m} + \dots + a_1 x^{e_1}, B(x) = b_n x^{f_n} + \dots + b_1 x^{f_1}$$

where

$$a_i, b_i \neq 0 \quad \text{and} \quad e_m > \dots > e_1 \geq 0, f_n > \dots > f_1 \geq 0.$$

Then clearly the number of coefficient additions can vary as

$$0 \leq \text{coefficient additions} \leq \min \{m, n\}.$$

- The **lower bound** is achieved when **none of the exponents are equal**, while the **upper bound** is achieved when the exponents of one polynomial are a **subset** of the exponents of the other.
- As for exponent comparisons, one comparison is made on each iteration of the **while** loop of lines 3-15.
- On each iteration either **p or q or both** move to the **next term** in their respective polynomials.
- Since the total no of terms is **$m + n$** , the **no of iterations** and hence the **number of exponent** comparisons is bounded by **$m + n$** .
- In summary then, the maximum number of executions of any of the statements in PADD is bounded above by **$m + n$** .
- Therefore, the computing time is **$O(m + n)$**

- The use of linked lists is well suited to polynomial operations.
- We can easily imagine writing a collection of procedures for input, output addition, subtraction and multiplication of polynomials using linked lists as the means of representation.
- A hypothetical user wishing to read in polynomials $A(x)$, $B(x)$ and $C(x)$ and then compute $D(x) = A(x) * B(x) + C(x)$ would write in his main program:

```

call READ(A)
call READ(B)
call READ(C)
 $T \leftarrow PMUL(A, B)$ 
 $D \leftarrow PADD(T, C)$ 
call PRINT(D)

```

- Youtube References
- For Polynomial Representation Refer:
 - <https://www.youtube.com/watch?v=hM-rvpvJ4Po>
- For Polynomial Addition
https://www.youtube.com/watch?v=V_ZNKu_pUPQ
- <https://gist.github.com/jamesgeorge007/336622f348815f0bef589bae5b622cff>

- Now user may wish to continue computing more polynomials.
- At this point it would be useful to **reclaim** the nodes which are being used to represent $T(x)$.
- This polynomial was created only as a **partial result** towards the answer $D(x)$.
- By returning the nodes of $T(x)$, they may be used to hold other polynomials.

procedure *ERASE*(*T*)

 //return all the nodes of *T* to the available space list avoiding repeated
 calls to procedure *RET* //

if *T* = 0 **then return**

p ← *T*

while *LINK*(*p*) ≠ 0 **do** //find the end of *T* //

p ← *LINK*(*p*)

end

LINK(*p*) ← *AV* // *p* points to the last node of *T* //

AV ← *T* //available list now includes *T* //

end *ERASE*

- This algorithm cleverly avoids using the **RET procedure** to return the nodes of *T* one node at a time, but makes use of the fact that the nodes of *T* are already linked.
- The **time** required to erase *T*(*x*) is still **proportional** to the **number of nodes** in *T*.
- This erasing of entire polynomials can be carried out **even more efficiently** by modifying the list structure so that the **LINK field of the last node points back to the first node** as in figure 4.8.
- A list in which the last node points back to the first will be termed a **circular list**.
- A **chain is a singly linked list** in which the last node has a zero link field.

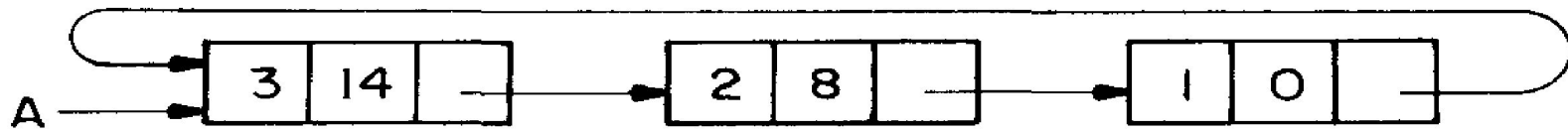


Figure 4.8 Circular List Representation of $A = 3x^{14} + 2x^8 + 1$

- Circular lists may be erased in a fixed amount of time **independent** of the **number of nodes in the list**.

procedure *CERASE*(*T*)

 // return the circular list *T* to the available pool //

if *T* = 0 **then return**;

$X \leftarrow \text{LINK}(T)$

$\text{LINK}(T) \leftarrow \text{AV}$

$\text{AV} \leftarrow X$

end *CERASE*

Figure 4.9 is a schematic showing the link changes involved in erasing a circular list.

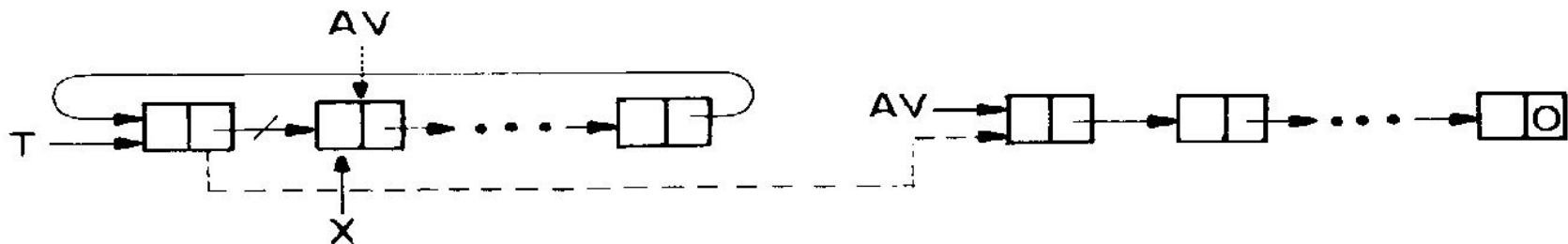
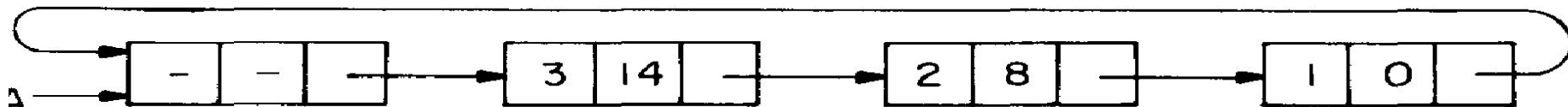


Figure 4.9 Dashes Indicate Changes Involved in Erasing a Circular List

- A **direct changeover** to the structure of (CLL)figure 4.8 however, causes some problems during addition, etc., as the **zero polynomial** has to be handled as a **special case**.
- To **avoid** such special cases one may introduce a **head node** into each polynomial; i.e., each polynomial, zero or non-zero, will contain one additional node.
- The **EXP** and **COEF** fields of this node will **not** be **relevant**.
- Thus the zero polynomial will have the representation:



while $A = 3x^{14} + 2x^8 + 1$ will have the representation



- For this circular list with head node representation the test for $T = 0$ may be removed from CERASE.
- The only changes to be made to algorithm PADD are:
 - (i) at line 1 define p, q by $p \leftarrow LINK(A); q \leftarrow LINK(B)$
 - (ii) at line 3: **while** $p \neq A$ **and** $q \neq B$ **do**
 - (iii) at line 16: **while** $p \neq A$ **do**
 - (iv) at line 20: **while** $q \neq B$ **do**
 - (v) at line 24: replace this line by $LINK(d) \leftarrow C$
 - (vi) delete line 25
- Thus the algorithm stays essentially the same.
- Zero polynomials are now handled in the same way as nonzero polynomials.
- A further simplification in the addition algorithm is possible if the EXP field of the head node is set to -1.
- Now when all nodes of A have been examined $p = A$ and $EXP(p) = -1$.
- Since -1 $EXP(q)$ the remaining terms of B can be copied by further executions of the case statement.
- The same is true if all nodes of B are examined before those of A
- This implies that there is no need for additional code to copy the remaining terms as in PADD.
- The final algorithm takes the following simple form.

procedure *CPADD* (*A*, *B*, *C*)

 //polynomials *A* and *B* are represented as circular lists with head nodes so that $EXP(A) = EXP(B) = -1$. *C* is returned as their sum, represented as a circular list//

$p \leftarrow LINK(A)$; $q \leftarrow LINK(B)$

call *GETNODE*(*C*); $EXP(C) \leftarrow -1$ // set up head node //

$d \leftarrow C$ // last node in *C* //

loop

case

 : $EXP(p) = EXP(q)$: **if** $EXP(p) = -1$ **then** [$LINK(d) \leftarrow C$; **return**]

$x \leftarrow COEF(p) + COEF(q)$

if $x \neq 0$ **then call** *ATTACH*($x, EXP(p), d$)

$p \leftarrow LINK(p)$; $q \leftarrow LINK(q)$

 : $EXP(p) < EXP(q)$: **call** *ATTACH*($COEF(q), EXP(q), d$)

$q \leftarrow LINK(q)$

 : **else** : **call** *ATTACH*($COEF(p), EXP(p), d$)

$p \leftarrow LINK(p)$

end

forever

end *CPADD*

- Let us review what we have done so far. We have introduced the notion of a **singly linked list**.
- Each element on the list is a node of fixed size containing 2 or more fields one of which is a link field.
- To represent many lists all within the same block of storage we created a special list called the **available space list** or *AV*.
- This list contains all nodes which are **currently not in use**.
- Since **all insertions and deletions for AV** are made at the **front**, what we really have is a **linked list being used as a stack**.